

64Forth

****Public domain.****

****64Forth's Heritage****: When I decided to make yet another Forth system, I went looking for a name for it. I thought of my earlier Forths, like F-PC, and Win32Forth, and thought possibly of Win64Forth. But this Forth is not designed for Windows, so that seemed wrong. I then thought of 64Forth, and went looking for Forth systems on the internet with that name. You will never guess what I found. Yes, you guessed it. 64Forth was the name of my earlier Forth system for the Commodore 64 Computer. In that case, the 64 represented the fact that the Commodore had I believe 64 MB of memory, which was quite a lot in that day. Anyway, I realized that I essentially already had dibs on the 64Forth name, so that is the name I chose for this MacOS M1-M5+ Forth system that is a hybrid of the two previous Forth systems I created this month. I hope you will find 64Forth interesting, at least enough to take a look. It is constructed mostly by Grok with an assembly language kernel, and a Swift code console and extensions, and like TZForth, it has some Libraries built right into the app. ****As of v0.9.0**** it includes File-Access, file-backed Blocks, Floating-point (`VOCABULARY FP`), Core/Core Ext, String Ext, Locals, ****Facility + Facility Ext**** (structures, `EKEY>FKEY`, `K-\*`), ****FacilityTerminal**** full-screen `PAGE`/`AT-XY`, ****SZ-EDITOR**** (FROMLIB + open panel, Cmd-S/W/Q), ****Extended Character (UTF-8 XChar)****, ****hashed multi-thread wordlists**** (`DICT\_THREADS`), a green Hayes subset (core through FP/paranoia Excellent), and modular Library ****ANSValidate**** (~383 passed / 0 failed). Remaining gaps vs full TZForth are mainly store sandbox and further editor polish. The architecture is very interesting: CODE words are assembly labels; macros build traditional headers (NFA, LFA, FFA, CFA, BODY) plus an HFA (Help Field Address). Go Forth and prosper!

****Getting 64Forth to run on your Mac****: All of the latest security changes Apple has made to MacOS, have made it fairly difficult to run apps obtained from outside the Apple App Store, but it is not impossible. Here is how you to it;

1. ****Download the .dmg**** file from the '****releases****' folder in the 64Forth development folder.
2. ****Open**** and view the .jpg image called '****Getting 64Forth to run.jpg****'.
3. This image shows a collage of the dialogs you have to traverse to get the MacOS to allow you to open the app.
4. Don't despair, it's not that hard, just follow along;
5. ****Mount the .dmg**** file and you will see ****64Forth.app****.
6. Drag the app onto your desktop.
7. Hold down the ****Control key**** and click the app and select ****Open**** from the menu that pops up.
8. You will get an error dialog that tells you that the app cannot be verified and will not be opened.
9. This last step is important because it sets up the MacOS so that you can now go into ****Settings**** and tell it to allow the app to open.
10. ****Open Settings****, and scroll down to ****Privacy & Security****. A list of apps and setting will be displayed.
11. Scroll down to the bottom of the ****Privacy & Security**** panel and you will see under the ****Security**** heading where it says '****64Forth.app****' ****was blocked to protect your Mac****.

12. To the right of the above message you will see a button **"Open Anyway"**. Click the button.
13. After clicking Open Anyway, another dialog will pop up that says basically **Trash, Open Anyway and Done. Click Open Anyway.**
14. After you click **Open Anyway** in that dialog, **another dialog will pop up and ask you for your password.** This is the final system dialog that is keeping you from running 64Forth. Simply type in your **"Mac" password,** and 64Forth will open and display it's **Opening screen.**
15. You are done, you can now run 64Forth without having to go through this again.

64Forth is a **macOS SwiftUI console app** whose **execution engine** is the **PickleForth ARM64 assembly kernel**, while the **host** (console, menus, Resources layout, FROMLIB-style library paths) follows **TZForth**.

		Terminal kernel		Full Swift Forth		Hybrid (this project)	
	--	-----	-----	-----	-----		
	Project		PickleForth		TZForth		**64Forth**
	UI		Terminal		SwiftUI console		SwiftUI console
	Engine		`forth.s` ITC		Swift `TZForth` class		`forth.s` ITC (embed)
	Libraries		cwd / absolute		`Resources/Library` + FROMLIB		same as TZForth (target)

See **DESIGN.md** for the integration plan and phases.

Layout

```

```text
64Forth/
 DESIGN.md
 README.md
 64Forth.xcodeproj/
 64Forth/
 App/ SwiftUI entry + content
 Host/ FileHost, FileAccess, BigIntHost, FloatHost,
 FacilityTerminal, KernelBridge
 Kernel/ forth.s, boot_words.inc, colon_words.inc, kernel_api.h
 Resources/ AutoLoad/, Library/, Docs/ → copied into app bundle
 Assets.xcassets/
...

```

---

## ## Status (v0.9.0)

- [x] Project folder and design doc
- [x] Kernel sources copied from PickleForth (`\_kernel\_cold\_start` entry, not `\_main`)
- [x] SwiftUI app shell + console
- [x] Resource folders (AutoLoad sample, Library/BigInteger & PI samples)
- [x] Embeddable `\_kernel\_init` / `\_kernel\_eval` / host EMIT·KEY (Phase 1)
- [x] Console parity: protected region, Return commit, ↑/↓ history, Tools menus (Phase 2)

- [x] FROMLIB + host-driven INCLUDE/FLOAD/REQUIRE (Phase 3)
- [x] AutoLoad on launch (`Resources/AutoLoad/autoload.fth` → MAIN) (Phase 4)
- [x] DIR + Phase 5 hardening (reenetrancy, bookmarks, entitlements; batched emit)
- [x] Search-Order vocabularies + BIG-INTEGER + host BI-MUL/DIVMOD/ISQRT + ALLOCATE
- [x] Locals (`{:` / `TO`) for full `big-int.fth`
- [x] `REQUIRED` / absolute-path include registry / `.INCLUDED`
- [x] Multi-wordlist `FORGET`, `RESIZE`, quoted `INCLUDE "...` paths, console KEY
- [x] File-Access word set (`FileAccess` host + CODE)
- [x] Block word set with file volume; `LOAD` restores outer `BLK`
- [x] Floating-point (`FloatHost` + `VOCABULARY FP`; `ALSO FP` to use)
- [x] In-app Hayes subset green: core through FP (`FROMLIB FLOAD HayesTest/HayesTest.fth`)
- [x] v0.6: FIND-before-number, Tools `?`, CODE `DEPTH`, include buffer 256\_KiB, Hayes harness
- [x] String Ext, Locals Ext, Facility structures, Extended Character (UTF-8 XChar)
- [x] \*\*v0.7.0:\*\* Modular ANS-VALIDATE (~351/0); Facility Ext (`EKEY>FKEY`, `K-\*`); `LOAD`/`BLK` restore; docs
- [x] \*\*v0.8.0:\*\* `FacilityTerminal` cell grid for `PAGE`/`AT-XY`; \*\*SZ-EDITOR\*\* full-screen edit (FROMLIB, open panel, Cmd-S/W/Q, Ctrl-Home/End); `S"/`.`" keep leading spaces after the name blank
- [x] \*\*v0.8.1:\*\* `ABORT` compile path uses `-2 POSTPONE LITERAL` (fixes memory fault when compiling e.g. `BI-ENSURE`)
- [x] \*\*v0.8.2:\*\* ANSValidate `host.fth` (TZForth FTEST high-ROI ports); `SLITERAL` cell-length fix
- [x] \*\*v0.9.0:\*\* Multi-thread wordlists (`DICT\_THREADS` = 16, hash in `\_header\_build`/FIND); `.THREADS` / `.VOCABULARIES`; `LAST` for `IMMEDIATE`/`ALIAS`/`RECURSE`/`MARKER`; prompt `ok(n)>` with data-stack depth; File-Access multi-result stack fix (`FILE\_POP\_UNDER`); Tools → Show Library/AutoLoad/Docs opens Finder via `NSWorkspace.open`; ANSValidate ~383/0 + stack hygiene; Hayes fail banners clearer

Optional later: line-at-a-time INCLUDE via fileid, App Sandbox for store builds, editor search/dual-buffer.

Open `64Forth.xcodeproj` in \*\*full Xcode\*\* (Apple Silicon; not Command Line Tools alone). Build the \*\*64Forth\*\* app target.

### ### Quick examples

```
```text
FROMLIB FLOAD BigInteger/big-int.fth
FROMLIB FLOAD HayesTest/HayesTest.fth
ALSO FP
1.5e0 2e0 F+ F.

FROMLIB FLOAD ANSValidate/ANS-VALIDATE.fth

FROMLIB FLOAD Editor/SZ-EDITOR.fth
FROMLIB SZEDIT Editor/SZ-EDITOR-README.txt
```
```

### ### Phase 1 API (assembly ↔ Swift)

| Symbol                     | Role                                                 |
|----------------------------|------------------------------------------------------|
| ----- -----                |                                                      |
| `kernel_init()`            | Boot dict + `forth_init_str`; returns (no QUIT loop) |
| `kernel_eval(line, n)`     | Interpret one line; `" ok\n"` via emit               |
| `kernel_set_emit(fn)`      | Host character output                                |
| `kernel_set_key(fn)`       | Host KEY (-1 if none)                                |
| `kernel_set_fromlib(fn)`   | Host arms FROMLIB resolve                            |
| `kernel_set_load_file(fn)` | Host reads file for INCLUDE/FLOAD                    |

**\*\*FROMLIB example\*\*** (after rebuild):

```
```text
FROMLIB FLOAD BigInteger/big-int.fth
```
```

(Resolves under `Resources/Library`. BigInteger + PI libraries and the Hayes suite are supported on the current kernel/host.)

See `64Forth/Kernel/kernel\_api.h`. Do **\*\*not\*\*** call `\_kernel\_cold\_start` from the UI.

---

### ## Chatting with Grok (important)

You are currently able to open Grok in **\*\*any project root\*\***. Treat **\*\*one primary folder per chat\*\***:

| Work on...                                    | Start Grok / open project from...       |
|-----------------------------------------------|-----------------------------------------|
| ----- -----                                   |                                         |
| <b>**64Forth**</b> (this hybrid app)          | `~/Documents/XCodeProjects/64Forth`     |
| <b>**PickleForth**</b> (terminal kernel only) | `~/Documents/XCodeProjects/PickleForth` |
| <b>**TZForth**</b> (Swift engine reference)   | `~/Documents/XCodeProjects/TZForth`     |

### **\*\*Recommendations:\*\***

1. **\*\*New chat when you switch products\*\*** – e.g. finish a PickleForth kernel chat, then open a **\*\*new\*\*** Grok session with `cwd = `64Forth``. That keeps commits, paths, and context from mixing.
2. **\*\*Point Grok at 64Forth\*\*** by opening that folder in the tool / starting the session from that directory (same as you did for PickleForth).
3. You can still **\*\*read\*\*** PickleForth or TZForth from a 64Forth chat when integrating (`../PickleForth`, `../TZForth`); prefer **\*\*writing\*\*** only under `64Forth` unless you explicitly want a PickleForth fix.
4. Kernel improvements: implement/test in PickleForth when pure assembly is easier, then **\*\*copy or cherry-pick\*\*** into `64Forth/64Forth/Kernel/`.

Use a **\*\*64Forth-rooted\*\*** Grok session for hybrid-app work; keep PickleForth sessions for pure terminal-kernel experiments.

---

## ## Origins

- Kernel lineage: PickleForth (ITC ARM64).
- Host/UX lineage: TZForth (SwiftUI console, Resources, FROMLIB concepts).
- Both public domain; 64Forth is public domain as well.